

1. מה ההבדל בין קריאת מערכת (System Call) לקריאה לפונקציה רגילה? איך זה עובד?

- **החלפת מצב (Mode Switch):** בניגוד לפונקציה רגילה שמתבצעת באותו מרחב זיכרון ובאותה רמת הרשאה, קריאת מערכת דורשת מעבר מ-"User Mode" (מצב משתמש) ל-"Kernel Mode" (מצב ליבה).
- **מנגנון ה-Trap:** התהליך משתמש בהוראת מעבד מיוחדת (ב-RISC-V זו הוראת ecall) כדי לייצר "מלכודת" (Trap) שמעבירה את השליטה לקרנל בנקודה מוגדרת מראש.
- **בידוד והגנה:** פונקציה רגילה היא חלק מקוד המשתמש, בעוד קריאת מערכת היא ממשק מבוקר המאפשר למשתמש לבקש שירותים מהחומרה או מהקרנל בצורה מאובטחת.

2. איך עוברים פרמטרים לקריאת המערכת? איך מוחזר ערך החזרה למשתמש?

- **שימוש באוגרים (Registers):** ב-xv6-riscv, הפרמטרים מועברים דרך אוגרי המעבד (בדרך כלל אוגרים a0, a1 וכו').
- **שליפת פרמטרים בקרנל:** הקרנל משתמש בפונקציות עזר (כמו argint או argaddr) כדי לקרוא את הערכים האלו מתוך מבנה ה-Trapframe של התהליך.
- **ערך חזרה:** ערך החזרה של הקריאה נכתב חזרה לאוגר a0 ב-Trapframe, כך שכשהתהליך חוזר ל-"User Mode", הערך נמצא באוגר הנכון עבור פונקציית ה-C שקראה לו.

3. מהי המטרה של הקובץ `usys.pl`?

- **יצירת Stubs:** זהו סקריפט Perl שמייצר באופן אוטומטי קובץ Assembly המכיל את ה-"Stubs" (עטיפות קצרות) עבור כל קריאות המערכת.
- **גשר למערכת:** כל Stub כזה מכיל את הקוד הדרוש כדי לשים את מספר קריאת המערכת (למשל SYS_memsz) באוגר המתאים ולבצע את הוראת ה-ecall.

4. מהו struct proc ואיפה הוא מוגדר? למה אנחנו צריכים אותו? האם למערכת הפעלה אמיתית יש מבנה דומה?

- **PCB (Process Control Block):** זהו מבנה הנתונים המרכזי המייצג תהליך בקרנל. הוא מאחסן

מידע קריטי כמו ה-PID, מצב התהליך (Running, Sleeping וכו'), גודל הזיכרון (sz), וטבלת הדפים שלו.

- **מיקום:** הוא מוגדר בקובץ `kernel/proc.h`.
- **מערכות אמיתיות:** כן, לכל מערכת הפעלה יש מבנה כזה. בלינוקס, למשל, הוא נקרא `struct task_struct`.

5. כמה זיכרון התוכנית צורכת לפני ואחרי ההקצאה?

- **לפני:** לפי תוצאות הריצה שלך, התוכנית צרכה **16,384 בתים**.
- **אחרי:** לאחר הקצאת 20,000 בתים, הזיכרון קפץ ל-**81,920 בתים**.

6. מה ההבדל בין גודל הזיכרון לפני ואחרי השחרור (release/free)?

- **אין הבדל:** לפי תוצאות הריצה, גודל הזיכרון נשאר **81,920 בתים** גם לאחר הקריאה ל-`free`.

7. הסבר את ההבדל (או חוסר ההבדל) לפני ואחרי השחרור. מה יכול לגרום לכך?

- **מנגנון ה-Malloc/Free:** פונקציית `free` ב-`User Mode` משחררת את הזיכרון לשימוש פנימי של התוכנית (מוסיפה אותו ל-`"Free List"` של הספרייה), אך היא לא בהכרח מבקשת מהקרנל להקטין את מרחב הכתובות של התהליך.
 - **הקטנת ה-Heap:** כדי להקטין את הזיכרון בפועל בקרנל, יש לקרוא לקריאת מערכת שמקטינה את ה-`sz` (כמו `sbrk` עם ערך שלילי). ספריית ה-C של `x64` לרוב לא עושה זאת עבור כל `free` כדי לשמור על ביצועים.
-